Design Document Batavia Nova Digital Bank

This document serves to describe how the Application Architecture of the Batavia Nova Digital Bank will be using a domain driven development approach

Authors	Sebastiaan van Uggelen
Date	10-03-2026
Version	0.1

1. Idea + Scope **6**
1.1 Idea 6
1.2 Scope 6
1.2.1 Ledger Context 6
1.2.2 Identity Context 6
1.3 Out of scope 6
2. Ubiquitous Language **7**
2.1 Money & Currency 7
2.2 Accounts 7
2.3 Balance 7
2.4 Transactions & Postings 7
2.5 Overdraft 8
2.5.1 Overdraft Exposure 8
2.5.2 Overdraft Validation Rules 8
2.6 Policies & Rules 9
2.7 Identity Context Terms 9
3. Bounded Contexts and Context maps **10**
3.1 Ledger context 10
3.1.1 Responsibilities 10
3.1.2 Core invariants 10
3.1.3 Main concepts 10
3.2 Identity context 11
3.2.1 Responsibilities 11
3.2.2 Core invariants 11
3.2.3 Main concepts 11
3.3 Context relationship 11
3.4 Context Map 12
3.5 Integration rules 12
4. Aggregates, Entities, Value Object and Domain Services **13**
4.1 Design approach 13
4.2 Ledger Context building blocks 13
4.2.1 Aggregate: Account 13
4.2.2 Entity: Posting 14
4.3.3 Aggregate: Ledger Transaction 15
4.3 Identity Context building blocks 16
4.3.1 Aggregate: UserIdentity 16
4.4 Value Objects 17
4.5 Domain Services 18
4.5.1 OverdraftPolicyService 18
4.5.2 BalanceCalculationService 18
4.5.3 StatementService 19
4.6 Aggregate interaction pattern 19
4.7 Class diagram 20

5. Commands, Domain Events, and Use Case Flows **21**
5.1 Command design principles 21
5.2 Command catalog 22
5.2.1 CreateCustomerAccount 22
5.2.2 CreateSystemAccount 23
5.2.3 DepositMoney 24
5.2.4 WithdrawMoney 25
5.2.5 TransferMoney 26
5.2.6 RequestOverdraftLimit 27
5.2.7 ReverseTransaction 28
5.3 Domain events 29
5.4 Event payload guidance 29
5.5 Use case flows 30
5.5.1 Flow: Create customer account 30
5.5.2 Flow: Deposit money 30
5.5.3 Flow: Withdraw money 30
5.5.4 Flow: Transfer money 31
5.5.5 Flow: Change overdraft limit 31
5.6 Validation ownership 32
5.7 Failure scenarios 32
5.8 Concurrency and consistency rule 33
5.9 User flow diagram 33
6. User Stories and Acceptance Criteria **34**
6.1 Epic: Identity and Access 34
US-001 Log in 34
US-002 View only owned customer accounts 34
6.2 Epic: Account Management 35
US-003 Create customer account 35
US-004 Create system account 35
6.3 Epic: Money Movements 36
US-005 Deposit money into a customer account 36
US-006 Withdraw money from a customer account 36
US-007 Transfer money to another account 37
US-008 Reverse a transaction 37
6.4 Epic: Balances and Statements 38
US-009 View current account balance 38
US-010 View account statement 38
US-011 View historical balance as of a point in time 39
6.5 Epic: Overdraft Management 40
US-012 Request account overdraft limit change 40
6.6 Epic: Audit and Invariants 41
US-013 Persist only balanced transactions 41
US-014 Prevent mutation of recorded transactions 41
6.7 Suggested implementation order 42

1. Idea + Scope

1.1 Idea

Building a banking app with a small double-entry ledger core that lets users record money movements between accounts in a way that is always internally consistent and auditable. Every movement of value is captured as a balanced transaction (postings sum to zero), so money can't be created or destroyed by accident, and balances are derived from history.

Users authenticate via username + password using an Identity solution (e.g., Duende IdentityServer). . Ledger accounts can be owned by a UserIdentity.

1.2 Scope

1.2.1 Ledger Context

- Create accounts (customer + system accounts like CashClearing)
- Record transactions for: deposit, withdrawal, transfer
- Store and show transaction history (account statement)
- Compute balance as derived state from postings (as-of now / as-of time)
- Strong invariants: balanced per currency, immutability of recorded transactions, single currency constraint (Money includes currency)
- Allow negative balances with an overdraft limit policy (simple rule)

1.2.2 Identity Context

- Create UserIdentity (username + password; Duende IdentityServer or similar)
- Link UserIdentity to one or more Customer accounts (system accounts cannot be user-owned)

1.3 Out of scope

- Payment requests / "betaalverzoek" / IOUs / scheduling intents
- External bank/payment integrations
- Interest, fees, regulatory/KYC/AML

2. Ubiquitous Language

2.1 Money & Currency

- Currency: A supported ISO currency code (e.g., `EUR`).
- Money: Value object `{ amount, currency }`.
- Single-Currency Account Invariant:
  An Account has exactly one `currency`. All Postings to that account must use the same currency.

2.2 Accounts

- Account: A ledger account with:
  - AccountId
  - Currency
  - AccountType: `Customer` | `System`
- Customer Account: Can be linked to exactly one UserIdentity as owner.
- System Account: Never user-owned (e.g., `CashClearing`).

2.3 Balance

- Balance: Derived by summing all postings for an account.
- Available Balance: Balance after applying overdraft rules (i.e., how far below zero you're allowed to go).

2.4 Transactions & Postings

- Transaction (immutable record of an intent like deposit/withdraw/transfer)
- Posting (the line items in a transaction that hit specific accounts)
- Balanced Transaction (the invariant that postings sum to zero)
- Reversal Transaction (how you correct mistakes without editing history)
- Statement (how transactions/postings are presented to a user)

2.5 Overdraft

- Account Overdraft Limit: A per-customer-account setting defining the *minimum allowed balance* for that account.
  - Example: `accountOverdraftLimit = 500 EUR` means balance may go down to `-500 EUR`.
- User Overdraft Cap scope (MVP)
  - Since the system is single-currency overall, the User Overdraft Cap is a single value (no per-currency variant):
    - `UserOverdraftCap` (in the system currency) User Cap is for configuration, not transaction posting

2.5.1 Overdraft Exposure

- Overdraft Used (per account):
  - `overdraftUsed = max(0, -balance)` (in that account's currency)
- Total Overdraft Used (per user, per currency):Sum of `overdraftUsed` across all customer accounts owned by the user in the same currency.
  - (Since accounts are single-currency, aggregate is naturally per currency.)

2.5.2 Overdraft Validation Rules

- Account Limit Check: After a proposed transaction, the account balance must be `>= -accountOverdraftLimit`.
- Overdraft Allocation: The portion of a user's cap assigned to a specific customer account as its `AccountOverdraftLimit` When transactions exceeds
- Reserved Overdraft (per user): `sum(AccountOverdraftLimit)` across all customer accounts owned by the user.
- Allocation Invariant: `ReservedOverdraft <= UserOverdraftCap`.
- Overdraft Change Command
  - RequestOverdraftLimit(accountId, newLimit): A user-initiated request to set/increase an account's overdraft limit.
  - Validation: reject outright if `newReservedOverdraft > userCap`.

2.6 Policies & Rules

- Overdraft Limit: A policy that allows a customer account balance to go negative down to some configured limit.
- Overdraft Policy Check: Validation that runs before posting a transaction that would reduce an account below its allowed limit.
- Authorization Rule: A user can only view/post against customer accounts they own (system accounts are restricted).

2.7 Identity Context Terms

- UserIdentity: Authenticated principal (username + password) managed by the identity provider.
- Identity Provider (IdP): Duende IdentityServer (or equivalent) that issues tokens for the app.
- Account Link: Association between a UserIdentity and a Customer Account.
- System Account Ownership Constraint: System accounts have no UserIdentity owner and cannot be linked.

3. Bounded Contexts and Context maps

After establishing the Ubiquitous Language, the next step is to define the bounded contexts in which that language applies. For Batavia Nova Digital Bank, the model is intentionally small and split into two primary bounded contexts: **Ledger** and **Identity**.

3.1 Ledger context

The Ledger Context is responsible for recording and validating movements of money. It is the source of truth for financial history and balances.

3.1.1 Responsibilities

- Manage ledger accounts
- Record balanced transactions and postings
- Enforce accounting invariants
- Derive balances from posting history
- Apply overdraft validation rules
- Provide account statements and transaction history

3.1.2 Core invariants

- Every transaction must be balanced
- Transactions are immutable once recorded
- Postings on an account must match the account currency
- Customer accounts may not exceed their configured overdraft limit

3.1.3 Main concepts

- Account
- Transaction
- Posting
- Balance
- Overdraft Limit
- Statement

3.2 Identity context

The Identity Context is responsible for authentication and ownership relations between authenticated users and customer accounts.

3.2.1 Responsibilities

- Register and authenticate users
- Manage credentials through an identity provider
- Maintain account ownership links for customer accounts
- Provide identity claims to the application

3.2.2 Core invariants

- A system account can never be linked to a user
- A customer account must have at most one owner in the MVP

3.2.3 Main concepts

- UserIdentity
- Identity Provider
- Account Link

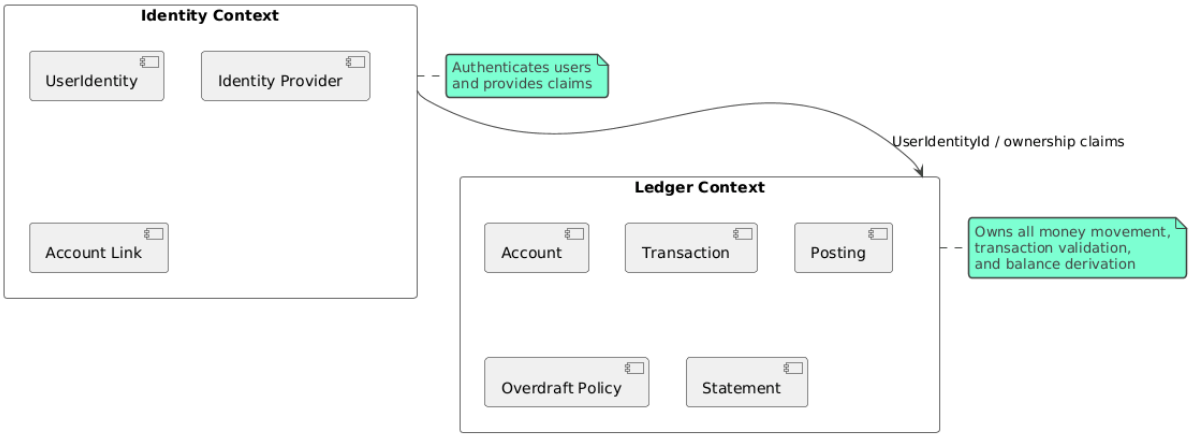
3.3 Context relationship

The relationship between the contexts is intentionally simple:

- Identity is responsible for who the user is
- Ledger is responsible for what happens to money
- The Ledger Context does not manage passwords or authentication flows
- The Identity Context does not make accounting decisions or compute balances

The connection between both contexts is made through a stable identifier such as `UserIdentityId`

3.4 Context Map



3.5 Integration rules

The integration between Identity and Ledger should happen through application services, not by sharing internal domain objects across contexts.

Examples:

- Ledger receives authenticated `UserIdentityId` from the application layer
- Ledger checks whether the user is allowed to act on the target customer account
- Identity never directly creates ledger transactions

4. Aggregates, Entities, Value Object and Domain Services

This section defines the tactical DDD building blocks inside the bounded contexts. The goal is to make explicit where invariants are enforced, which objects hold identity, and which rules belong in services rather than in individual entities.

4.1 Design approach

The Batavia Nova MVP should keep aggregates small and focused on consistency boundaries. In particular:

- account state and account policy must be validated consistently
- transaction recording must guarantee balanced postings
- identity and authentication concerns must stay outside the ledger model

A practical design for the MVP is therefore:

- Account as an aggregate for account-specific rules
- LedgerTransaction as an aggregate for immutable financial events
- UserIdentity as an aggregate in the Identity Context
- Domain services for rules that span multiple aggregates, such as total overdraft allocation checks

4.2 Ledger Context building blocks

4.2.1 Aggregate: Account

The Account aggregate is the consistency boundary for account-specific constraints.

Responsibilities

- Hold account identity and classification
- Enforce single-currency invariant
- Hold overdraft configuration for customer accounts
- Determine whether a debit would violate account-level overdraft policy
- Guard ownership-related constraints relevant to the ledger model

Suggested fields

- `AccountId`
- `Currency`
- `AccountType` (`Customer` | `System`)
- `OwnerUserIdentityId` (nullable for system accounts)
- `AccountOverdraftLimit`
- `Status` (`Active` | `Blocked` | `Closed`)

Invariants

- Account currency is immutable after creation
- System accounts cannot have an owner
- Customer accounts can have at most one owner in the MVP
- `AccountOverdraftLimit >= 0`
- Account postings must always use the account currency

Notes

The Account aggregate should not store the full running balance as the source of truth in the domain model if the design principle is “balance derived from history.” It may expose balance-related behavior through domain services or projections, but the authoritative source remains posting history.

4.2.2 Entity: Posting

A Posting is an entity within a transaction aggregate. It does not live independently.

Responsibilities

- Represent one ledger line against one account
- Carry signed monetary impact for that account
- Link the transaction to affected accounts

Suggested fields

- `PostingId`
- `AccountId`
- `Money Amount`
- `Direction` (`Debit` | `Credit`) or signed amount representation
- `BookingTime`
- `Description` (optional, limited)

Invariants

- Posting currency must match the target account currency
- Posting must belong to exactly one transaction
- Posting cannot be changed after the transaction is recorded

4.3.3 Aggregate: Ledger Transaction

The LedgerTransaction aggregate is the consistency boundary for immutable recording of financial intent.

Responsibilities

- Group postings into one atomic financial event
- Enforce balanced transaction invariant
- Preserve immutable audit history
- Carry transaction metadata needed for statements and reversals

Suggested fields

- `TransactionId`
- `TransactionType` (`Deposit` | `Withdrawal` | `Transfer` | `Reversal`)
- `CreatedAt`
- `CreatedByUserIdentityId` (nullable for system-originated transactions)
- `Reference`
- `ReversalOfTransactionId` (nullable)
- `Postings[]`

Invariants

- A transaction must contain at least two postings
- Sum of postings must equal zero per currency
- Transaction is immutable once persisted
- Reversal transaction must reference an earlier transaction
- Reversal creates compensating postings; it does not edit history

Notes

This aggregate should be created only after all validation is complete. That means the application layer prepares the command, loads required account information, invokes domain rules, and then constructs a valid immutable transaction.

4.3 Identity Context building blocks

4.3.1 Aggregate: UserIdentity

The UserIdentity aggregate belongs to the Identity Context and should remain separate from ledger rules.

Responsibilities

- Represent the authenticated user
- Hold identity-related identifiers used by the application
- Support ownership linkage to customer accounts through integration boundaries

Suggested fields

- `UserIdentityId`
- `Username`
- `IdentityProviderSubject`
- `Status`

Invariants

- Username must be unique within the Identity Context
- Authentication credentials are managed by the identity provider, not the ledger

4.4 Value Objects

The following concepts should be modeled as value objects because they are defined by their attributes and have no lifecycle identity of their own.

Money

- Fields: `Amount`, `Currency`
- Rules:
  - amount precision must match currency rules chosen by the system
  - arithmetic is only valid for equal currencies
  - immutable by design

Currency

- Fields: `Code`
- Rules:
  - must be a supported ISO currency code
  - immutable by design

Balance

- Represents derived state, not persisted as the primary source of truth in the domain model
- May include:
  - `CurrentBalance`
  - `AvailableBalance`
  - `AsOfTimestamp`

OverdraftLimit

- Encapsulates the allowed negative exposure for an account
- Rules:
  - cannot be negative
  - must use the account currency

AccountOwnership

- Represents the link between a customer account and a `UserIdentityId`
- Useful if you want to avoid primitive obsession around raw IDs in the application layer

4.5 Domain Services

Some rules do not fit naturally inside one aggregate because they require information across multiple aggregates. Those rules should be modeled as domain services.

4.5.1 OverdraftPolicyService

This domain service evaluates whether a proposed transaction is allowed with respect to overdraft rules.

Responsibilities

- Check account-level overdraft limit after a proposed debit
- Check user-level overdraft cap allocation when account limits are created or changed
- Separate pure business rules from persistence and transport concerns

Example decisions handled here

- Would this withdrawal make account balance fall below `-AccountOverdraftLimit`?
- Would changing an account overdraft limit make total reserved overdraft exceed `UserOverdraftCap`?

Inputs

- target account
- current derived balance
- proposed debit amount
- user overdraft allocation state

Outputs

- allow / reject
- structured business reason for rejection

4.5.2 BalanceCalculationService

This domain service derives balances from posting history.

Responsibilities

- Calculate account balance as-of now
- Calculate account balance as-of a historical timestamp
- Calculate overdraft used from derived balance

Notes

This keeps balance derivation explicit and testable. It also avoids embedding historical summation logic inside the Account aggregate itself.

4.5.3 StatementService

This service prepares statement-oriented domain views from transactions and postings.

Responsibilities

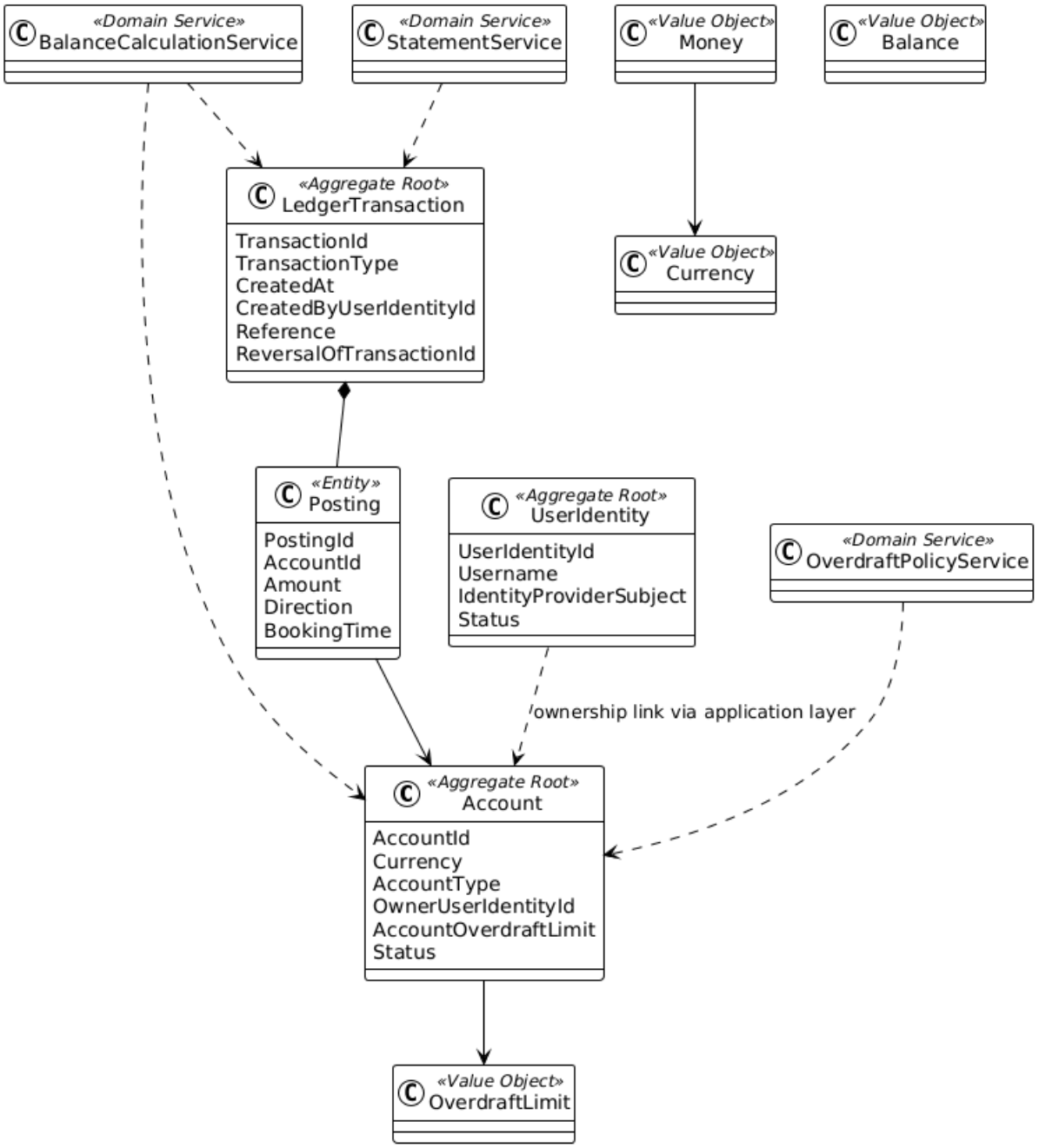
- Convert internal ledger events into account statement lines
- Preserve transaction references and timestamps
- Support pagination and as-of filtering in the application layer

4.6 Aggregate interaction pattern

For the MVP, commands should follow this pattern:

1. Authenticate user in the Identity Context
2. Authorize account access in the application layer using ownership information
3. Load affected accounts and relevant derived balances
4. Run overdraft and currency validation
5. Construct a valid immutable `LedgerTransaction`
6. Persist the transaction and expose updated projections

4.7 Class diagram



5. Commands, Domain Events, and Use Case Flows

This section translates the domain model into executable behavior. It defines which commands can be issued to the system, which validation rules apply, which domain events result from successful execution, and how the main MVP use cases flow through the application.

The purpose of this section is not to define HTTP APIs yet, but to make the behavioral contract of the system explicit so implementation can begin.

5.1 Command design principles

For the Batavia Nova MVP, commands should follow these rules:

- A command expresses an intention to change state
- A command is validated before any ledger transaction is recorded
- A successful money movement results in exactly one immutable `LedgerTransaction`
- A failed command records no financial state change
- Commands that span multiple accounts are handled atomically
- Authentication and authorization happen in the application layer before domain execution

5.2 Command catalog

5.2.1 CreateCustomerAccount

Creates a customer ledger account for a user.

Input

- `AccountId`
- `OwnerUserIdentityId`
- `Currency`
- `InitialOverdraftLimit` (optional, default `0`)

Validation

- user identity must exist
- currency must be supported
- account id must be unique
- account type must be `Customer`
- initial overdraft limit must be `>= 0`

Result

- customer account is created and linked to the owner

Domain event

- `CustomerAccountCreated`

5.2.2 CreateSystemAccount

Creates a system ledger account such as `CashClearing`.

Input

- `AccountId`
- `Currency`
- `SystemAccountName`

Validation

- currency must be supported
- account id must be unique
- system account must not have an owner

Result

- system account is created

Domain event

- `SystemAccountCreated`

5.2.3 DepositMoney

Records a deposit into a customer account by posting value from a system account into the customer account.

Input

- `TargetAccountId`
- `SourceSystemAccountId`
- `Amount`
- `InitiatedByUserIdentityId`
- `Reference`

Validation

- target account must exist
- source system account must exist
- initiated user must be authorized for the target account
- amount must be positive
- currencies of both accounts and amount must match
- source account and target account must be different
- postings must be constructible as a balanced transaction

Result

- immutable deposit transaction is recorded

Domain event

- `MoneyDeposited`
- `LedgerTransactionRecorded`

5.2.4 WithdrawMoney

Records a withdrawal from a customer account into a system account.

Input

- SourceAccountId
- TargetSystemAccountId
- Amount
- InitiatedByUserIdentityId
- Reference

Validation

- source account must exist
- target system account must exist
- initiated user must be authorized for the source account
- amount must be positive
- currencies of both accounts and amount must match
- resulting source account balance must remain `>= -AccountOverdraftLimit`
- postings must be constructible as a balanced transaction

Result

- immutable withdrawal transaction is recorded

Domain event

- MoneyWithdrawn
- LedgerTransactionRecorded

5.2.5 TransferMoney

Records a transfer between two customer accounts.

Input

- `FromAccountId`
- `ToAccountId`
- `Amount`
- `InitiatedByUserIdentityId`
- `Reference`

Validation

- both accounts must exist
- initiated user must be authorized for the source account
- amount must be positive
- source and target accounts must be different
- currencies of both accounts and amount must match
- resulting source account balance must remain `>= -AccountOverdraftLimit`
- postings must be constructible as a balanced transaction

Result

- immutable transfer transaction is recorded

Domain event

- `MoneyTransferred`
- `LedgerTransactionRecorded`

5.2.6 RequestOverdraftLimit

Requests a change to the overdraft limit of a customer account.

Input

- `AccountId`
- `RequestedLimit`
- `InitiatedByUserIdentityId`

Validation

- account must exist
- account must be a customer account
- initiated user must be authorized for the account
- requested limit must be `>= 0`
- new reserved overdraft across all owned accounts must not exceed `UserOverdraftCap`

Result

- account overdraft limit is updated

Domain event

- `OverdraftLimitChanged`

5.2.7 ReverseTransaction

Creates a compensating transaction for a previously recorded transaction.

Input

- `TransactionId`
- `InitiatedByUserIdentityId`
- `Reason`

Validation

- original transaction must exist
- original transaction must not already be reversed if that is the chosen MVP rule
- reversal must reproduce opposite postings in the same currency
- user must be authorized according to operational policy

Result

- compensating reversal transaction is recorded

Domain event

- `TransactionReversed`
- `LedgerTransactionRecorded`

5.3 Domain events

Domain events represent facts that already happened in the domain. In the MVP they are useful for auditing, projections, statements, and integration between write and read models.

Account events

- `CustomerAccountCreated`
- `SystemAccountCreated`
- `OverdraftLimitChanged`

Transaction events

- `MoneyDeposited`
- `MoneyWithdrawn`
- `MoneyTransferred`
- `TransactionReversed`
- `LedgerTransactionRecorded`

5.4 Event payload guidance

Each domain event should contain enough information to support auditing and read-model updates without forcing downstream consumers to re-run business logic.

Suggested common event fields:

- `EventId`
- `OccurredAt`
- `AggregateId`
- `InitiatedByUserIdentityId` (nullable for system actions)
- `CorrelationId`
- `CausationId`

Suggested transaction event fields:

- `TransactionId`
- `TransactionType`
- `PostingSummaries`
- `Reference`

5.5 Use case flows

5.5.1 Flow: Create customer account

1. User is authenticated by the Identity Context
2. Application receives `CreateCustomerAccount`
3. Application validates input and owner identity
4. `Account` aggregate is created with type `Customer`
5. Ownership link is stored
6. `CustomerAccountCreated` is emitted
7. Read model is updated so the account becomes visible to the user

5.5.2 Flow: Deposit money

1. User is authenticated
2. Application receives `DepositMoney`
3. Authorization checks that the user may act on the target customer account
4. Application loads target account and source system account
5. Currency and positivity rules are validated
6. A balanced `LedgerTransaction` is constructed with two postings
7. Transaction is persisted atomically
8. `MoneyDeposited` and `LedgerTransactionRecorded` are emitted
9. Balance and statement projections are updated

5.5.3 Flow: Withdraw money

1. User is authenticated
2. Application receives `WithdrawMoney`
3. Authorization checks that the user owns the source customer account
4. Application loads source account, target system account, and derived current balance
5. Currency and positivity rules are validated
6. `OverdraftPolicyService` checks resulting account balance
7. A balanced `LedgerTransaction` is constructed with two postings
8. Transaction is persisted atomically
9. `MoneyWithdrawn` and `LedgerTransactionRecorded` are emitted
10. Balance and statement projections are updated

5.5.4 Flow: Transfer money

1. User is authenticated
2. Application receives `TransferMoney`
3. Authorization checks that the user owns the source account
4. Application loads both accounts and the source derived balance
5. Currency and positivity rules are validated
6. `OverdraftPolicyService` validates source account overdraft rule
7. A balanced `LedgerTransaction` is constructed
8. Transaction is persisted atomically
9. `MoneyTransferred` and `LedgerTransactionRecorded` are emitted
10. Balance and statement projections are updated for both accounts

5.5.5 Flow: Change overdraft limit

1. User is authenticated
2. Application receives `RequestOverdraftLimit`
3. Authorization checks that the user owns the account
4. Application loads all customer accounts of the user relevant to reserved overdraft calculation
5. `OverdraftPolicyService` checks new reserved overdraft against `UserOverdraftCap`
6. Account overdraft limit is updated
7. `OverdraftLimitChanged` is emitted
8. Read model is updated

5.6 Validation ownership

To prevent an anemic or overly coupled model, validation should be split by responsibility.

Application layer validation

- authentication
- authorization
- command shape and required fields
- loading required aggregates and projections
- orchestration of atomic persistence

Domain validation

- balanced transaction invariant
- account currency invariant
- overdraft limit rules
- system account ownership constraint
- reversal correctness

5.7 Failure scenarios

The MVP should explicitly reject the following command outcomes:

- account not found
- unauthorized account access
- unsupported currency
- amount less than or equal to zero
- source and target account are identical where disallowed
- overdraft limit exceeded
- overdraft allocation cap exceeded
- unbalanced transaction construction
- attempt to mutate immutable historical transaction
- attempt to assign owner to a system account

These should return structured business errors rather than technical exceptions where possible.

5.8 Concurrency and consistency rule

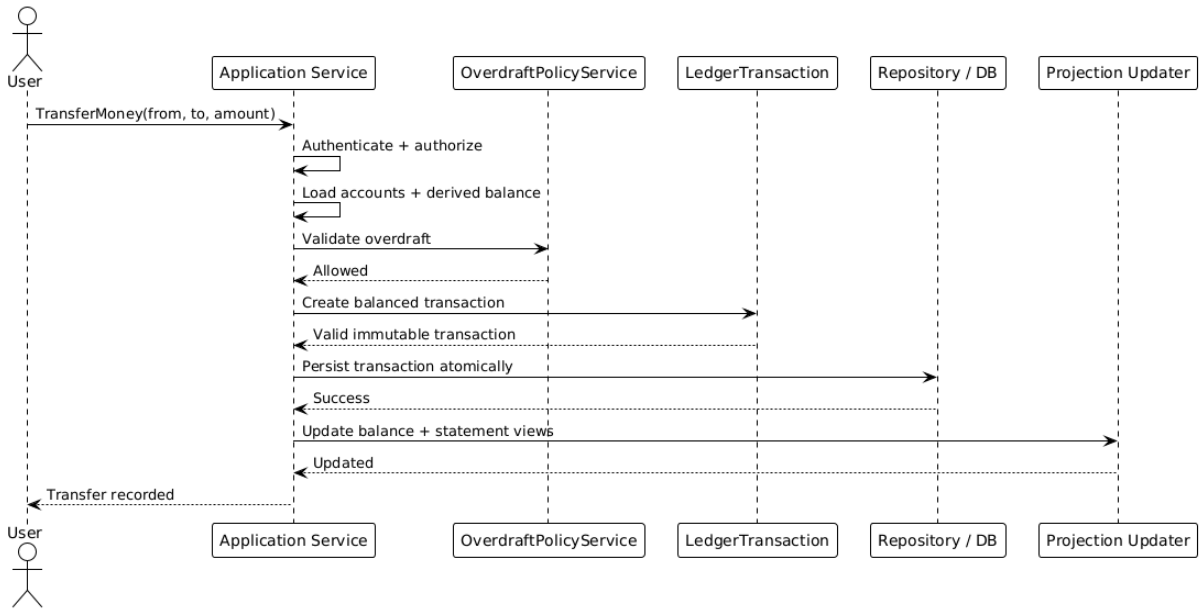
For the MVP, each successful command that records a money movement must commit atomically together with its postings. The system must not allow a partially persisted transaction.

A practical implementation rule is:

- load the required state
- validate using the latest committed data
- persist the new immutable transaction in one database transaction
- update projections after the write succeeds

If optimistic concurrency is introduced, the command should fail and be retried by the caller or application workflow when the underlying state changed during validation.

5.9 User flow diagram



6. User Stories and Acceptance Criteria

This section converts the domain model and command flows into concrete MVP user stories that can be implemented by a development team. The stories are intentionally small and vertically sliced where possible.

6.1 Epic: Identity and Access

US-001 Log in

As a user
I want to log in with my username and password
So that I can access my bank accounts securely.

Acceptance Criteria

- Given a registered user with valid credentials, when the user submits the correct username and password, then access is granted.
- Given invalid credentials, when the user attempts to log in, then access is denied.
- Given a successful login, when access is granted, then the application receives an authenticated `UserIdentityId`.
- Given an unauthenticated request to a protected ledger function, when the request is made, then the request is rejected.

US-002 View only owned customer accounts

As a user
I want to see only the customer accounts I own
So that I cannot access accounts belonging to others.

Acceptance Criteria

- Given an authenticated user, when the user requests the account overview, then only customer accounts linked to that user are returned.
- Given a system account, when the account overview is shown to a normal user, then the system account is not shown unless explicitly allowed by role.
- Given a user tries to access another user’s customer account, when the request is made, then the request is rejected as unauthorized.

6.2 Epic: Account Management

US-003 Create customer account

As a user
I want to create a customer account in the supported currency
So that I can hold and move money in the system.

Acceptance Criteria

- Given an authenticated user, when the user requests account creation with a supported currency, then a new customer account is created.
- Given a new customer account is created, when creation succeeds, then the account is linked to the requesting user.
- Given an unsupported currency, when account creation is requested, then the request is rejected.
- Given an initial overdraft limit is provided, when the account is created, then the overdraft limit is stored if valid.
- Given an invalid initial overdraft limit smaller than zero, when account creation is requested, then the request is rejected.

US-004 Create system account

As an administrator or system operator
I want to create a system account such as `CashClearing`
So that ledger transactions can balance against internal accounts.

Acceptance Criteria

- Given an authorized operator, when a supported system account is requested, then the system account is created.
- Given a system account is created, when creation succeeds, then the account has no owner.
- Given a normal user tries to create a system account, when the request is made, then the request is rejected.

6.3 Epic: Money Movements

US-005 Deposit money into a customer account

As a user
I want to deposit money into my account
So that my account balance increases.

Acceptance Criteria

- Given an authenticated user who owns the target account, when the user submits a positive deposit amount, then a balanced ledger transaction is recorded.
- Given a successful deposit, when the transaction is stored, then the account balance increases by the deposited amount.
- Given a successful deposit, when the transaction is stored, then the transaction appears in the account statement.
- Given a zero or negative amount, when a deposit is requested, then the request is rejected.
- Given a currency mismatch between the amount and involved accounts, when a deposit is requested, then the request is rejected.

US-006 Withdraw money from a customer account

As a user
I want to withdraw money from my account
So that my account balance decreases within allowed overdraft rules.

Acceptance Criteria

- Given an authenticated user who owns the source account, when the user submits a valid positive withdrawal amount, then a balanced ledger transaction is recorded.
- Given a successful withdrawal, when the transaction is stored, then the account balance decreases by the withdrawn amount.
- Given the withdrawal would cause the balance to fall below `-AccountOverdraftLimit`, when the withdrawal is requested, then the request is rejected.
- Given a successful withdrawal, when the transaction is stored, then the transaction appears in the account statement.
- Given a zero or negative amount, when a withdrawal is requested, then the request is rejected.

US-007 Transfer money to another account

As a user
I want to transfer money from my account to another account
So that I can move value between accounts in the ledger.

Acceptance Criteria

- Given an authenticated user who owns the source account, when the user submits a valid transfer to a different target account, then a balanced ledger transaction is recorded.
- Given a successful transfer, when the transaction is stored, then the source account balance decreases and the target account balance increases by the same amount.
- Given the transfer would violate the source account overdraft limit, when the transfer is requested, then the request is rejected.
- Given source and target account are the same, when the transfer is requested, then the request is rejected.
- Given a currency mismatch, when the transfer is requested, then the request is rejected.
- Given a successful transfer, when the transaction is stored, then the transfer appears in the statements of both accounts.

US-008 Reverse a transaction

As an authorized operator
I want to reverse an incorrect transaction without editing history
So that the ledger remains auditable and correctable.

Acceptance Criteria

- Given an existing transaction, when an authorized operator requests reversal, then a compensating transaction is recorded.
- Given a successful reversal, when the reversal transaction is stored, then the original transaction remains unchanged.
- Given the reversal succeeds, when account statements are viewed, then both the original transaction and the reversal are visible.
- Given an unauthorized user requests reversal, when the request is made, then the request is rejected.

6.4 Epic: Balances and Statements

US-009 View current account balance

As a user
I want to view my current account balance
So that I know how much money is available.

Acceptance Criteria

- Given an authenticated user who owns the account, when the account details are requested, then the current derived balance is shown.
- Given an account with overdraft configured, when the balance is shown, then the available balance reflects the overdraft rule.
- Given an unauthorized user requests the balance of another user's account, when the request is made, then the request is rejected.

US-010 View account statement

As a user
I want to view the transaction history of my account
So that I can audit movements of money.

Acceptance Criteria

- Given an authenticated user who owns the account, when the statement is requested, then transactions affecting that account are returned in chronological order.
- Given an account with multiple transactions, when the statement is requested, then each statement line includes at least timestamp, reference, amount impact, and resulting context needed for display.
- Given pagination is applied, when the user requests a page of statement lines, then the results are returned consistently.
- Given an unauthorized user requests another user's statement, when the request is made, then the request is rejected.

US-011 View historical balance as of a point in time

As a user
I want to view my account balance at a past moment
So that I can understand historical account state.

Acceptance Criteria

- Given an authenticated user who owns the account, when an as-of timestamp is provided, then the balance is derived from postings up to that time.
- Given no transactions existed before the requested timestamp, when the historical balance is requested, then zero is returned.
- Given an unauthorized user requests a historical balance for another user's account, when the request is made, then the request is rejected.

6.5 Epic: Overdraft Management

US-012 Request account overdraft limit change

As a user
I want to request a change to my account overdraft limit
So that I can allocate my available overdraft capacity to the account.

Acceptance Criteria

- Given an authenticated user who owns the account, when a valid non-negative overdraft limit is requested, then the request is evaluated.
- Given the resulting reserved overdraft across the user's accounts stays within `UserOverdraftCap`, when the change is requested, then the account overdraft limit is updated.
- Given the resulting reserved overdraft would exceed `UserOverdraftCap`, when the change is requested, then the request is rejected.
- Given a negative requested limit, when the request is made, then the request is rejected.

6.6 Epic: Audit and Invariants

US-013 Persist only balanced transactions

As a system
I want every recorded money movement to be balanced
So that the ledger remains internally consistent.

Acceptance Criteria

- Given a deposit, withdrawal, transfer, or reversal, when the transaction is recorded, then postings sum to zero per currency.
- Given a transaction cannot be constructed as balanced, when persistence is attempted, then the transaction is rejected.
- Given a successful transaction, when it is stored, then no partial postings exist without the parent transaction.

US-014 Prevent mutation of recorded transactions

As a system
I want recorded transactions to be immutable
So that the ledger remains auditable.

Acceptance Criteria

- Given a persisted transaction, when an update to its postings is attempted, then the request is rejected.
- Given an error in a recorded transaction, when correction is needed, then reversal is used instead of mutation.

6.7 Suggested implementation order

The following order gives a good first vertical slice and fast feedback:

1. `US-001` Log in
2. `US-003` Create customer account
3. `US-005` Deposit money into a customer account
4. `US-009` View current account balance
5. `US-010` View account statement
6. `US-006` Withdraw money from a customer account
7. `US-007` Transfer money to another account
8. `US-012` Request account overdraft limit change
9. `US-008` Reverse a transaction
10. `US-011` View historical balance

7. Definition of Done and Test strategy